

Programação *assíncrona*

Boa parte do tempo de um programa é **espera**: pela rede, por um banco de dados, por um arquivo. Com `async` e `await`, o Python aproveita essas esperas para adiantar outras tarefas, num único fluxo de execução. Nesta aula: corrotinas, `asyncio.run`, rodar tarefas ao mesmo tempo com `gather` e `TaskGroup`, o erro de usar `time.sleep`, limites de tempo e o `async for`.

`async def` · `await`

`asyncio.run`

`gather`

`time.sleep` × `asyncio.sleep`

`TaskGroup`

`timeout`

`async for`

Formação Claude Code 2026 · Baseado em docs.python.org (asyncio: Coroutines and Tasks, Runners, Task Groups, Timeouts; Reference: `async for`, asynchronous generators), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em [exemplos/](#) · Saídas reais, com tempos arredondados

async def e await

primeira.py

```
import asyncio

async def saudar(nome):           # função assíncrona (corrotina)
    await asyncio.sleep(0.1)      # espera sem travar o programa
    return f"Olá, {nome}!"

c = saudar("Ana")
print(type(c).__name__)          # chamar só cria a corrotina
print(asyncio.run(c))            # o asyncio.run executa
```

SAÍDA

```
coroutine
Olá, Ana!
```

- ✓ `async def` cria uma **corrotina**: chamar não roda o corpo, só cria o objeto (como um gerador)
- ✓ `await` espera o resultado de outra corrotina e, enquanto isso, libera o programa
- ✓ `asyncio.run()` liga o **loop de eventos** e roda a corrotina principal

`await` só pode ser usado dentro de uma função `async def`.

await em sequência: *sem ganho*

sequencial.py

```
import asyncio
import time

async def baixar(arquivo, segundos):
    print(f"  início {arquivo}")
    await asyncio.sleep(segundos)      # simula a rede
    print(f"  fim {arquivo}")
    return arquivo

async def main():
    inicio = time.perf_counter()
    await baixar("a.pdf", 1)            # um depois do outro
    await baixar("b.pdf", 1)
    print(f"total: {time.perf_counter() - inicio:.1f} s")

if __name__ == "__main__":
    asyncio.run(main())
```

SAÍDA

```
início a.pdf
fim a.pdf
início b.pdf
fim b.pdf
total: 2.0 s
```

- ✓ Cada `await` espera o anterior terminar: $1\text{ s} + 1\text{ s} = 2\text{ s}$
- ✓ `asyncio.sleep` simula uma espera de rede, sem baixar nada de verdade
- ✓ Escrever `async` não deixa nada mais rápido: o ganho vem de rodar junto

gather: *três esperas em uma*

concorrente.py

```
import asyncio
import time
from sequencial import baixar

async def main():
    inicio = time.perf_counter()
    resultados = await asyncio.gather(  # ao mesmo tempo
        baixar("a.pdf", 1),
        baixar("b.pdf", 1),
        baixar("c.pdf", 1),
    )
    print(resultados)
    print(f"total: {time.perf_counter() - inicio:.1f} s")

asyncio.run(main())
```

A. PDF
espera 1 s



B. PDF
espera 1 s



C. PDF
espera 1 s

- ✓ As três começam juntas e esperam juntas: total de 1 s, não 3 s
- ✓ `gather` devolve os resultados **na ordem em que foram passados**
- ✓ É **concorrência**: uma linha de execução alternando entre as tarefas nas esperas

SAÍDA

```
início a.pdf
início b.pdf
início c.pdf
fim a.pdf
fim b.pdf
fim c.pdf
['a.pdf', 'b.pdf', 'c.pdf']
total: 1.0 s
```

time.sleep *trava tudo*

bloqueio.py

```
import asyncio
import time

async def errado():
    time.sleep(0.5)           # trava o loop inteiro

async def certo():
    await asyncio.sleep(0.5)  # libera o loop enquanto espera

async def medir(funcao):
    inicio = time.perf_counter()
    await asyncio.gather(funcao(), funcao(), funcao())
    gasto = time.perf_counter() - inicio
    print(f"{funcao.__name__}: {gasto:.1f} s")

asyncio.run(medir(errado))
asyncio.run(medir(certo))
```

SAÍDA

```
errado: 1.5 s
certo: 0.5 s
```

- ✓ `time.sleep` não tem `await` : segura o loop e as tarefas esperam em fila
- ✓ `asyncio.sleep` devolve o controle ao loop durante a espera
- ✓ O mesmo vale para bibliotecas comuns de rede e banco: no código assíncrono, use as versões assíncronas

Uma única chamada bloqueante deixa o programa assíncrono tão lento quanto o sequencial.

TaskGroup: *tarefas em grupo*

tarefas.py

```
import asyncio

async def contar(nome, n):
    for i in range(1, n + 1):
        await asyncio.sleep(0.1)
        print(f"{nome} {i}")
    return f"{nome} terminou"

async def main():
    async with asyncio.TaskGroup() as grupo:      # Python 3.11+
        t1 = grupo.create_task(contar("A", 2))
        t2 = grupo.create_task(contar("B", 3))
    print(t1.result(), "|", t2.result())

asyncio.run(main())
```

SAÍDA

```
A 1
B 1
A 2
B 2
B 3
A terminou | B terminou
```

- ✓ `create_task` agenda a corrotina para rodar em paralelo às outras
- ✓ O bloco `async with` só termina quando **todas** as tarefas acabam
- ✓ Se uma falhar, o grupo cancela as outras e mostra o erro
- ✓ As linhas se intercalam: A e B avançam juntas

`TaskGroup` existe desde o Python 3.11 e é a forma recomendada de agrupar tarefas.

Desistir *de quem demora*

tempo_limite.py

```
import asyncio

async def consulta_lenta():
    await asyncio.sleep(2)
    return "dados"

async def main():
    try:
        async with asyncio.timeout(0.5):    # desiste em 0,5 s
            print(await consulta_lenta())
    except TimeoutError:
        print("A consulta demorou demais: cancelada")

asyncio.run(main())
```

SAÍDA

```
A consulta demorou demais: cancelada
```

- ✓ `asyncio.timeout(0.5)` cancela o que estiver dentro do bloco depois de 0,5 s
- ✓ O cancelamento vira um `TimeoutError`, que você trata como qualquer exceção
- ✓ Essencial para chamadas de rede: um servidor travado não trava o seu programa

Dados que *chegam aos poucos*

async_for.py

```
import asyncio

async def mensagens():                # gerador assíncrono
    for texto in ["oi", "tudo bem?", "tchau"]:
        await asyncio.sleep(0.1)      # chegam aos poucos
        yield texto

async def main():
    async for msg in mensagens():
        print("recebido:", msg)

asyncio.run(main())
```

SAÍDA

```
recebido: oi
recebido: tudo bem?
recebido: tchau
```

- ✓ Um `async def` com `yield` é um gerador assíncrono (aulas 03 e 04)
- ✓ `async for` espera cada item com `await`, sem travar o loop
- ✓ Uso típico: mensagens de chat, linhas de um stream, eventos de um websocket

async ajuda *em esperas, não em contas*

TIPO DE TRABALHO	EXEMPLOS	FERRAMENTA
Espera de E/S	APIs, downloads, banco de dados, sockets	<code>asyncio</code>
E/S com bibliotecas comuns	código bloqueante que não dá para trocar	<code>threads</code> (<code>asyncio.to_thread</code>)
Cálculo pesado	processar imagens, simulações, criptografia	<code>processos</code> (<code>multiprocessing</code>)

- ✓ O `asyncio` roda numa linha de execução só: ganha tempo **esperando várias coisas juntas**
- ✓ Uma conta demorada ocupa essa linha e ninguém mais anda
- ✓ Para scripts simples, sem muitas esperas, o código síncrono continua sendo o mais fácil de ler

Pergunta rápida: "meu programa fica parado esperando algo de fora?"
Se sim, `asyncio` pode ajudar.

Menor preço *entre três lojas*

Enunciado: consulte o preço em três lojas (com demoras diferentes) ao mesmo tempo e mostre o menor. O tempo total deve ser o da loja mais lenta, não a soma.

Uma resposta possível

```
import asyncio
import time

LOJAS = {"Loja A": (0.6, 199.9), "Loja B": (0.3, 189.0),
        "Loja C": (0.9, 205.5)}

async def consultar(loja):
    demora, preco = LOJAS[loja]
    await asyncio.sleep(demora)      # simula a API da loja
    return loja, preco

async def main():
    inicio = time.perf_counter()
    respostas = await asyncio.gather(*map(consultar, LOJAS))
    loja, preco = min(respostas, key=lambda r: r[1])
    print(f"menor preço: {loja} (R$ {preco:.2f})")
    print(f"tempo: {time.perf_counter() - inicio:.1f} s")

asyncio.run(main())
```

SAÍDA

```
menor preço: Loja B (R$ 189.00)
tempo: 0.9 s
```

- ✓ `map(consultar, LOJAS)` cria uma corrotina por loja; o `*` passa todas para o `gather`
- ✓ 0,9 s é a demora da Loja C, a mais lenta (a soma seria 1,8 s)
- ✓ Desafio: use `asyncio.timeout(0.5)` e ignore as lojas que demorarem mais

Lojas, demoras e preços são de exemplo.

Teste o que *ficou*

1. O que acontece ao chamar uma função async def sem await?

A) Roda normalmente

B) Só cria uma corrotina

C) Dá erro de sintaxe

D) Roda em outra thread

B.

2. Três tarefas de 1 s com asyncio.gather levam cerca de:

A) 3 s

B) 1 s

C) 0 s

D) 0,3 s

B.

3. Dentro de uma corrotina, para esperar 1 s sem travar as outras:

A) time.sleep(1)

B) await asyncio.sleep(1)

C) asyncio.sleep(1) sem await

D) input()

B.

4. Para qual trabalho o asyncio mais ajuda?

A) Muitas chamadas a APIs

B) Calcular primos enormes

C) Ordenar uma lista pequena

D) Somar dois números

A.

Esperar várias coisas *ao mesmo tempo*

`async def` cria corrotinas, `await` espera sem travar e `asyncio.run` liga o loop. O ganho vem de rodar junto: `gather` ou `TaskGroup`. Nada de `time.sleep` nem chamadas bloqueantes dentro delas; ponha limites com `asyncio.timeout` e consuma fluxos com `async for`. É para esperas de E/S, não para cálculo pesado.

`async` · `await`

`asyncio.run`

`gather` · `TaskGroup`

`timeout`

`async for`

Próxima aula → 12 · Usando Metaclasses

Fontes: docs.python.org/3/library/asyncio-task.html (Coroutines, Tasks, `gather`, `TaskGroup`, `timeout`, `to_thread`); docs.python.org/3/library/asyncio-runner.html; docs.python.org/3/reference/compound_stmts.html#async-for; docs.python.org/3/reference/expressions.html#asynchronous-generator-functions, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.